



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Department of Physics and Astronomy "A. Righi"

Theoretical Physics

Complex Networks

Lectures held by
Prof. Daniel Remondini

Notes by
Gioele Mancino

Academic Year 2025/2026

Contents

1	Definitions	3
1.1	Basic Graph Concepts	4
1.2	Network Representations	5
1.3	Graph Isomorphism and Permutation	9
1.4	Connectedness	11
1.5	Distances and Shortest Paths	13
1.6	Subgraphs and Special Network Topologies	16
1.7	Planarity and Homomorphism	22
1.8	Network Reduction and Filtering	23
1.9	Spectral Analysis of Networks	25
2	Random Graph Ensembles	31
2.1	Erdős-Rényi Random Graphs	32
2.2	Statistical Ensembles of Networks	33
2.3	Phase Transitions and the Giant Component	37
2.4	Network Properties in Random Graphs	38
2.5	Stochastic Block Model (SBM)	40
3	Spectral Analysis of Random Graphs	41
	Appendices	43
A	Notation and Conventions	43
B	Computations and further developments	43

Introduction

1 | Definitions

In this chapter, we will cover the basic definitions and concepts related to complex networks. We will start with fundamental graph concepts, then explore various network representations, discuss graph isomorphism and permutation, and finally delve into connectedness, distances, subgraphs, planarity, homomorphism, network reduction, and spectral analysis of networks.

1.1 | Basic Graph Concepts

We are going to start with the most basic definition of a graph, which serves as the foundation for all subsequent concepts in network theory. To introduce the concept of a graph, we will use the following definition:

Definition 1.1 (Graph). A graph $G(V, L)$ is a finite nonempty set V of p points (vertexes or nodes) and a set of k couples of distinct points L (links, lines, or edges).

From this definition, the concept of a graph is quite general and can be applied to a wide range of systems. The nodes can represent various entities such as people, computers, or proteins, while the links can represent relationships, interactions, or connections between these entities. The flexibility of this definition allows us to model complex systems in various fields, including social networks, communication networks, biological networks, and many others. There are some general rules that we typically follow when working with graphs, which are important to keep in mind as we explore more complex network structures.

Remark (General Rules).

- *In general, networks have no loops (self-connections).*
- *There are no multigraphs, meaning no multiple links between the same nodes.*
- *We can, however, consider multiple links as weights or as multiple layers (multiplex).*

1.1.1 | Directed and Undirected Graphs

Directed graphs (DiGraphs) are graphs with directed edges, known as arcs, indicating a “**from-to**” relationship. They are used to describe hierarchical relationships, fluxes, or the passing of messages and goods. Undirected graphs contain only symmetric links and are used to describe symmetrical relations such as friendship, chemical bonds, or statistical correlation. Note that directed graphs can be symmetrized; while this loses some information, it allows the use of simpler properties and algorithms.

1.1.2 | Walk, Trail, Path, and Cycle

Let us consider a graph with nodes i and j . We can define different types of connections between these nodes:

- **Walk:** A node and edge sequence between two nodes.
- **Trail:** A walk with all distinct links.
- **Path:** A walk with all distinct nodes and links.
- **Cycle:** A closed path with $N_c \geq 3$ nodes.

This classification is important for understanding the structure of networks and for defining concepts such as connectedness and distances. Different algorithms are used to find walks, trails, paths, and cycles in a graph, which can be crucial for applications like network traversal, routing, and community detection.

1.2 | Network Representations

It is important to note that a graph can be represented in various ways, each with its own advantages and disadvantages. The choice of representation often depends on the specific application and the properties of the graph being studied.

1.2.1 | Adjacency Matrix

The most common representation of a graph is the adjacency matrix, which is a square matrix that encodes the presence or absence of links between nodes. For unweighted graphs, the entries are binary (0 or 1), while for weighted graphs, the entries can take on a range of values corresponding to the weights of the links.

Definition 1.2 (Adjacency Matrix). The adjacency matrix A is an $N \times N$ matrix representing N nodes. For unweighted networks, $A_{ij} = 1$ indicates a topological link from node i to node j , with $A_{ii} = 0$ to indicate no loops. For weighted networks, $A_{ij} = w_{ij}$ where w_{ij} is the weight of the link.

Directed Graph

Consider the following directed graph with 6 nodes, we can represent it with the following adjacency matrix:

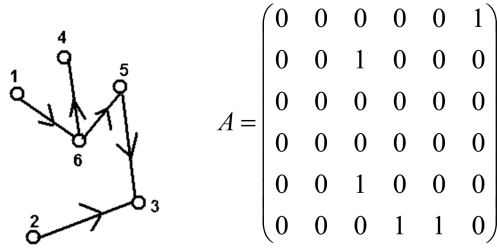


Figure 1.1: A directed graph and its corresponding non-symmetric adjacency matrix.

Undirected Graph

Consider the following undirected graph with 6 nodes, we can represent it with the following adjacency matrix:

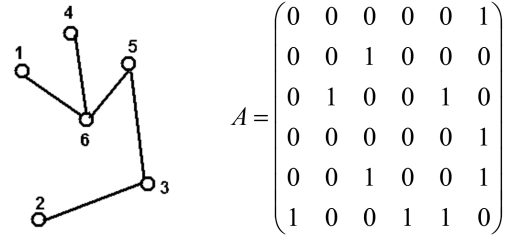


Figure 1.2: An undirected graph and its corresponding symmetric adjacency matrix.

Remark (Logical Operators on Adjacency Matrices). We can apply element-wise logical operators on adjacency matrices:

- Network symmetrization uses a logical OR: $A' = \max(A, A^T) = A \cup A^T$.
- Identifying reciprocal links uses a logical AND: $A'' = \min(A, A^T) = A \cap A^T$.
- Identifying strictly directed links uses logical XOR: $XOR(A, A^T)$.

1.2.2 | Weighted Network

In a weighted network, the adjacency matrix contains values that represent the strength of the connections between nodes:

$$A_{ij} = w_{ij} \text{ link from node } i \text{ to node } j \text{ with weight } w_{ij}.$$

This can be interpreted in various ways depending on the context, such as the number of interactions, the intensity of relationships, or the capacity of links.

It can be seen as a representation of a **multigraph** with N multi-edges: $w_{ij} = N$. If all weights are positive, we have useful theorems for matrices with positive values (Perron-Frobenius theorem) and we can connect to "physical" situations (transition probabilities of diffusion processes) Negative weights can be meaningful too: in **signed networks** the sign of the weight can represent different types of relationships or interactions, such as:

- **Friend-enemy**: in social networks, positive weights can represent friendships while negative weights can represent enmity.
- **Attraction-repulsion**: in physical systems, positive weights can represent attractive forces while negative weights can represent repulsive forces.
- **Ferromagnetic-antiferromagnetic**: in spin systems, positive weights can represent ferromagnetic interactions while negative weights can represent antiferromagnetic interactions.

One can also operate a weighted matrix symmetrization:

- Average symmetrization: $A' = \frac{1}{2}(A + A^T)$.
- Random matrix symmetrization: $A' = \frac{1}{\sqrt{2}}(A + A^T)$.

1.2.3 | Incidence Matrix

We can also represent a graph using an incidence matrix, which is an alternative to the adjacency matrix. The incidence matrix is particularly useful for certain types of analyses, such as flow problems and network dynamics.

Definition 1.3 (Incidence Matrix). The incidence matrix I of a graph $G(V, L)$ is an $L \times V$ matrix (number of links by number of vertexes). It contains -1 for outgoing connections and $+1$ for ingoing connections:

$$\text{if } A_{ij} \neq 0, \quad I_{Ni} = \mp 1, \quad I_{Nj} = \pm 1.$$

It can be interpreted as a "generalized derivative" on the graph, giving the oriented difference between values on adjacent nodes. If there is no preferred direction, we can use an unsigned incidence matrix with $I_{Ni} = 1$ for all links. It has to be read as if each link is a vector pointing from the source node to the target node, and the incidence matrix encodes the direction and magnitude of these vectors.

Thus the incidence matrix can be used to analyze the flow of information or resources through the network, and it is particularly useful for studying directed graphs and their properties. The incidence matrix can also be used to derive the adjacency matrix through the relation $A = I^T I - D$, where D is a diagonal matrix that accounts for the degree of each node.

Generalized Derivative

The incidence matrix can be interpreted as a generalized derivative on the graph, giving the oriented difference between values on adjacent nodes. The incidence matrix allows us to compute the net flow into or out of each node, which is essential for understanding the dynamics of processes.

Given a vector of values $\mathbf{f}_i$ for each node, the incidence matrix I applied on the vector gives:

$$\sum_j I_{ij} \cdot \mathbf{f}_j = \Delta_i,$$

$$\Delta_i = \mathbf{f}_i - \mathbf{f}_j \forall \text{adjacent nodes},$$

where Δ_i represents the net flow into or out of node i based on the values of its neighbors. This interpretation is crucial for analyzing diffusion processes, random walks, and other dynamic phenomena on networks.

Geometrically, a network acts as a spatial structure: while a regular lattice discretizes Euclidean space, a complex graph represents a more intricate manifold. However, one must be careful, as the properties of this discrete manifold do not necessarily hold in the continuous limit.

1.2.4 | Sparse Representations

For sparse graphs where the number of links is much smaller than V^2 , other representations are preferred:

- **Edge Matrix:** An $L \times 2$ matrix that stores single links instead of the full matrix. It is more efficient for sparse graphs, but it does not allow for fast access to neighbors. It is way less memory-consuming for sparse graphs:

$$L = o(V^2) \implies 2 \times L \ll V^2.$$

It is useful for algorithms that need to iterate over links, such as those used in community detection or clustering.

- **Adjacency List:** Each line describes a node and contains the indices of its arrival nodes. This is particularly useful for diffusion processes, such as simulating random walks on a graph, but it has a higher memory cost than the edge matrix, since it requires storing the neighbors of each node and the same link is stored twice (once for each node it connects).

Example. Sparse Graph Representations Let us consider a sparse graph with 6 nodes and 4 links. The adjacency matrix representation would require a 6×6 matrix, which contains many zeros, while the edge matrix would only require a 4×2 matrix to store the links. The adjacency list would also be more efficient in this case, as it would only need to store the neighbors of each node, rather than the full matrix. Explicitly:

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}, \quad I = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad E = \begin{pmatrix} 1 & 2 \\ 2 & 3 \\ 4 & 5 \\ 5 & 6 \end{pmatrix}, \quad L : \begin{array}{l} 1 \rightarrow \{2\} \\ 2 \rightarrow \{1, 3\} \\ 3 \rightarrow \{2\} \\ 4 \rightarrow \{5\} \\ 5 \rightarrow \{4, 6\} \\ 6 \rightarrow \{5\} \end{array}$$

where A is the adjacency matrix, I is the incidence matrix, E is the edge matrix, and L is the adjacency list. As we can see, the adjacency matrix contains many zeros, while the edge matrix and adjacency list are more compact representations of the same graph.

Random Walks and Diffusion Processes

The choice of representation can significantly impact the efficiency of algorithms for simulating random walks and diffusion processes on networks. For example, the adjacency list allows for efficient access to neighbors, which is crucial for simulating random walks, while the edge matrix can be more efficient for algorithms that need to iterate over links, such as those used in community detection or clustering.

To emulate a **random walk on a graph**, we can use the adjacency list to quickly access the neighbors of each node and determine the probabilities of transitioning from one node to another. This is particularly important for large, sparse graphs where the number of links is much smaller than V^2 , as it allows for efficient memory usage and faster computations. The steps to follow for simulating a random walk on a graph using the adjacency list are as follows:

1. Start at a randomly selected node (or a specific node if desired).
2. At each step, use the adjacency list to find the neighbors of the current node.
3. Randomly select one of the neighbors to transition to, based on the weights of the links (for a pure random walk we have to consider a uniform distribution, otherwise we could use the link weights/biases etc.).
4. Repeat the process for a specified number of steps or until a certain condition is met (e.g., reaching a target node).

This method allows us to efficiently simulate random walks and diffusion processes on large, sparse networks, which is essential for understanding the dynamics of complex systems.

1.3 | Graph Isomorphism and Permutation

We are now going to discuss the concept of graph isomorphism, which is a fundamental notion in graph theory that describes when two graphs are essentially the same, even if they are represented differently.

Definition 1.4 (Graph Isomorphism). An isomorphism is a 1-1 correspondence between vertexes that preserves the link structure. It corresponds to a node index permutation.

This means that two graphs are isomorphic if there exists a permutation of the node labels of one graph that results in the adjacency matrix of the other graph. In other words, the two graphs have the same structure, but their nodes may be labeled differently. We have then to digress on the concept of node permutation and permutation matrix, which is a fundamental operation in graph theory that allows us to rearrange the nodes of a graph without changing its structure.

A permutation of the nodes of a graph can be represented by a permutation matrix, which is a square binary matrix that has exactly one entry of 1 in each row and each column, and 0s elsewhere. The permutation matrix can be used to rearrange the rows and columns of the adjacency matrix of a graph, effectively permuting the nodes. There exist $N!$ possible permutations of the nodes, which means that there are $N!$ different permutation matrix realising the isomorphism: each $N \times N$ network can be represented by $N!$ different adjacency matrices, all isomorphic.

The properties of the permutation matrix are as follows:

$$\begin{aligned} P \cdot P^T &= \mathbb{I} \implies P^T = P^{-1} && \text{(orthogonality)} \\ P \cdot A \cdot P^T &= A' && \text{(adjacency matrix transformation)} \\ |P| &= \pm 1 && \text{(determinant)} \end{aligned}$$

Remark (Re-indexing). Node permutation represents node reordering, which is a common operation in graph algorithms. It allows us to rearrange the nodes of a graph without changing its structure, which can be useful for various applications such as community detection, clustering, and visualization. However, it is important to note that while node permutation can change the appearance of the adjacency matrix, it does not change the underlying properties of the graph, such as its degree distribution, clustering coefficient, or path lengths. Thus node re-indexing has to be made after gaining insights on the graph structure, thus it can be based on:

- **A priori information:** we can have information directly about the network, due to ontological knowledge of the system (e.g. known community structure or node attributes) or intrinsic metric properties of the graph (e.g., spatial networks, node proximity).
- **Network-based information:** we can have information about the network structure, such as node degree, centrality measures, or community structure, which can be used to reorder the nodes in a way that reveals patterns or simplifies the analysis.

These re-indexing strategies can help us to better understand the structure of the graph and to identify important features or patterns that may not be immediately apparent in the original representation. The **visualization** of a network can be significantly improved by reordering the nodes based on their properties or the structure of the graph, which can help to reveal hidden patterns and relationships between nodes (a chain network can be visualized as a sparse one if we make poor re-indexing choices).

Example (Protein and DNA Contact Maps). A network representation of peptide bonds serves as a discretization of a 3D distance matrix: a fully connected graph with weights given by the spatial distance between amino acids. The re-indexing of nodes can be based on the spatial proximity of amino acids, which can help to reveal patterns in the contact map and provide insights into the structure and function of the protein.

Spatial closeness acts as an “intrinsic” metric for the graph. One can use the re-indexing in order to exploit different properties of the graph, such as symmetry or centrality measures, study spatial closeness, revealing the backbone of the network, or identify communities.

DNA 3D structure can also be represented as a similar network, where nodes represent nucleotides and links represent spatial proximity or interactions between them. Re-indexing based on spatial proximity can help to reveal patterns in the contact map and provide insights into the structure and function of the DNA molecule (the hi-c technique is used to obtain such contact maps).

1.4 | Connectedness

Studying the connectedness of a graph is crucial for understanding its structure and the dynamics of processes that occur on it. Connectedness refers to the extent to which nodes in a graph are connected to each other, and it can be classified into different types based on the nature of the connections.

1.4.1 | Undirected Graphs

An undirected graph is **connected** if it is not divided into two or more non-communicating parts: from every node there is a path to every other node. If there are multiple disconnected components, the graph is called **disconnected**. Algebraically, this means there is no permutation that reduces the adjacency matrix into diagonal blocks, it is not isomorphic to a block diagonal matrix

$$PAP^{-1} = \begin{pmatrix} A_1 & 0 \\ 0 & A_2 \end{pmatrix}.$$

The number of blocks corresponds to the number of disconnected components. Any matrix which can be transformed into a block form (diagonal or upper triangular) is called **reducible**, while a matrix that cannot be transformed into such a form is called **irreducible**.

It is important to notice that from the same dataset more than one graph can be constructed, depending on the definition of the links; even from the same adjacency matrix, different re-indexing can lead to different visualizations of the same graph, which can reveal different patterns and structures. But before any analysis, it is crucial to check the connectedness of the graph, as many properties and algorithms assume that the graph is connected. Some network properties rely on the assumption of connectedness, other than the components number and size, so it is a crucial information to have before starting any analysis.

1.4.2 | Directed Graphs

In a directed graph, not every node is necessarily reachable from any other. After a **block matrix reduction**, we are often left with a block upper triangular matrix, where the blocks on the diagonal represent strongly connected components

$$PAP^{-1} = \begin{pmatrix} A_1 & B \\ 0 & A_2 \end{pmatrix}, \quad A_1 \xrightarrow{B} A_2, \quad A_1 \nleftarrow A_2.$$

In this situation, we can say that block A_1 is **upstream** of block A_2 , while block A_2 is **downstream** of block A_1 . The presence of the block B indicates that there are connections from nodes in block A_1 to nodes in block A_2 , but not vice versa. This structure can have important implications for the dynamics of processes on the graph, such as diffusion or information flow.

In a directed graph, connectedness does not guarantee that every node is reachable from any other. It is classified into three types:

- **Strongly connected:** Every node is connected to another by a path, implying the existence of cycles.
- **Unilaterally connected:** For every two nodes, there is at least one path (i.e. in one direction, as for the absorbing states seen before).

- **Weakly connected:** Every node is connected to another by at least a semipath.

Semi-paths, semi-walks etc. are defined as paths, walks etc. where it is necessary to traverse edges in an unspecified direction (we may need to reverse the direction of one or more links to complete the path). The presence of cycles in a directed graph is a key factor in determining whether the graph is strongly connected, as cycles allow for bidirectional connectivity between nodes.

We can also write three theorems about the connectedness of directed graphs:

- Strongly connected implies that exists one spanning closed walk.
- Unilaterally connected implies that exists one spanning walk.
- Weakly connected implies that exists one spanning semiwalk.

All these theorems can be proved by contradiction, by assuming that there is no spanning walk and showing that this leads to a contradiction with the definition of connectedness.

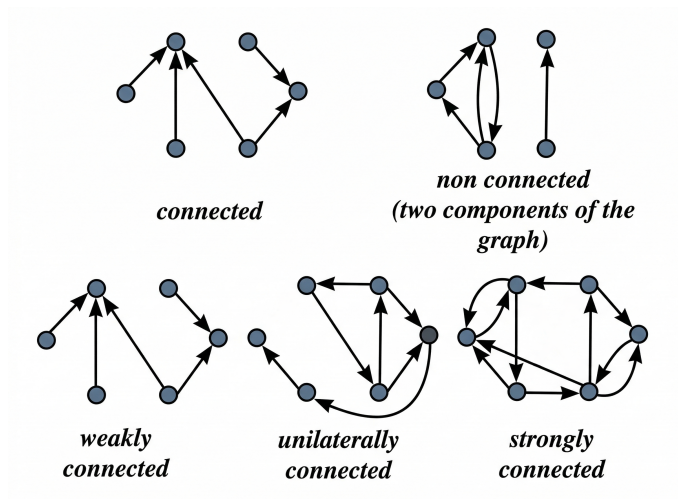


Figure 1.3: Examples of different types of connectedness in directed graphs. The left graph is weakly connected (since there is a semipath between every pair of nodes), the middle graph is unilaterally connected (since there is at least one path between every pair of nodes), and the right graph is strongly connected (since every node is reachable from every other node with a path).

1.5 | Distances and Shortest Paths

We are now going to discuss the concept of distance in graphs, which is a fundamental notion that allows us to quantify how far apart nodes are from each other. The distance between two nodes can be defined in various ways, depending on the context and the type of graph being studied.

1.5.1 | Node Distance

The distance between two nodes is defined as the shortest path between them. There are different ways to define distance, depending on the context and the type of graph:

- **Unweighted distance:** The distance is the number of links in the shortest path between two nodes.
- **Weighted distance:** The distance is the sum of the weights of the links in the shortest path between two nodes.

Let us study more in detail each case.

Unweighted Distance

In unweighted networks, the distance between two nodes is simply the number of links in the shortest path connecting them. This is often referred to as the **geodesic distance**. The geodesic distance can be defined as

- Isolated nodes have infinite distance from any other node: $D(u, v) = \infty$ if there is no path between nodes u and v .
- Adjacent nodes have a distance of 1: $D(u, v) = 1$ if there is a link between nodes u and v .
- A node is at distance 0 from itself: $D(u, v) = 0$ if $u = v$.
- The distance between two nodes is the minimum number of links in any path connecting them: $D(u, v) = \min\{k : \text{there exists a path of length } k \text{ from } u \text{ to } v\}$.

Note that topologically, if we are defining a distance on a symmetric graph, we are defining a metric space, since the distance satisfies the properties of a metric:

1. **Positive semidefiniteness:** $D(u, v) \geq 0$ for all nodes u and v , and $D(u, v) = 0$ if and only if $u = v$.
2. **Symmetry:** $D(u, v) = D(v, u)$ for all nodes u and v .
3. **Triangle inequality:** $D(u, w) \leq D(u, v) + D(v, w)$ for all nodes u , v , and w .

One last remark is that one can define different types of distances, such as the distance between node embeddings, one with the overlap of n neighbors, or the distance between communities, which can be defined as the distance between their centroids in a suitable embedding space.

Weighted Distance

In weighted networks, nodes that are “closer” have a greater weight, and distance can be calculated using **harmonic composition** (similar to capacitors in series): if we have more path between two nodes, the shortest path is the one with the greatest weight. We can define the weighted distance as

$$d_{ij} = \sum_{i \rightarrow j} \frac{1}{w_{kl}},$$

where the sum is taken over all paths from node i to node j , and w_{kl} is the weight of the link between nodes k and l along the path. This definition allows us to capture the idea that stronger connections (higher weights) correspond to shorter distances, while weaker connections (lower weights) correspond to longer distances. If there is a zero-weight link, it can be interpreted as an infinite weight (i.e., no connection), which means that the distance between the nodes is infinite:

$$w = 0 \implies d \rightarrow \infty.$$

Example. The matrix of **Euclidean distances** d_{ij} between elements of a defined N-dim space, can be seen as a fully connected symmetric weighted network: every element is connected to every other element, and the weight of the link between two elements is given by the Euclidean distance between them. This representation can be useful for analyzing spatial relationships and clustering in data, as well as for visualizing high-dimensional data in a lower-dimensional space. For example, in a protein structure, we can represent the distances between amino acids as a weighted network, where nodes represent amino acids and links represent the spatial proximity between them. This can help us to understand the structure and function of the protein, as well as to identify important interactions between amino acids.

Depending on the context, we can also define the distance as a function of the weights $d_{ij} = f(w_{ij})$, where f is a suitable function that captures the relationship between weights and distances. For example, we could use an exponential decay function, or a logarithmic function, one based on gaussian kernels, etc. The choice of the function f would depend on the specific application and the properties of the data being analyzed.

1.5.2 | Shortest Path and Algorithms

The distance between two nodes equals the minimum number of links (geodesic) between them. It is useful to define some auxiliary concepts:

- **Eccentricity:** The maximum distance from a node to any other node in the graph.
- **Radius:** The minimum eccentricity among all nodes in the graph.
- **Diameter:** The maximum eccentricity among all nodes in the graph.
- **Average path length:** The average distance between all pairs of nodes in the graph.

The shortest path can be found using various algorithms, some of the most common ones include:

- **Dijkstra’s algorithm:** This algorithm is used for *finding the shortest paths between nodes in a graph*, which may represent, for example, road networks. It works by iteratively selecting the node with the smallest tentative distance and updating the distances to its neighbors until the shortest path to the target node is found (one-to-all mechanism).

- **Floyd-Warshall algorithm:** This algorithm is used for *finding the shortest paths between all pairs of nodes in a graph*. It works by iteratively updating a distance matrix based on the shortest paths found so far, until all pairs of nodes have been considered (all-to-all mechanism).

Both of these algorithms are implemented efficiently in every SW language (such as Python, C, R, Matlab etc.) and can be used to compute shortest paths in large graphs. The choice of algorithm depends on the specific requirements of the problem, such as whether we need to find the shortest path between a single pair of nodes or between all pairs of nodes.

1.6 | Subgraphs and Special Network Topologies

We are going to discuss some special types of subgraphs and network topologies that are commonly studied in graph theory and network science. These structures can provide insights into the properties and dynamics of complex networks.

Definition 1.5 (Subgraph, Spanning Subgraph, Induced Subgraph). • **Subgraph:** A subset of a graph's nodes and links.

- **Spanning Subgraph:** A subgraph that contains all the nodes of the original graph (but not necessarily all the links).
- **Induced Subgraph:** A subset of the nodes of G with ALL the links of G that exist between those specific nodes.

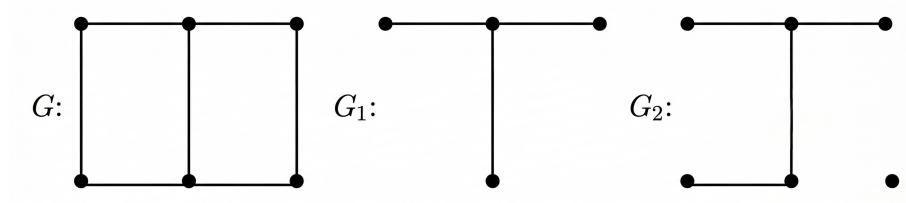


Figure 1.4: Examples of different types of subgraphs. G is the original graph, G_1 is an induced subgraph while G_2 is a spanning subgraph.

To extract the adjacency matrix of an induced subgraph S , you simply need to find the intersection of the rows and columns of the original matrix A that correspond exactly to the nodes in S . Practically, removing elements from the network translates to very basic matrix operations:

- **Node removal:** To remove a specific node i from the network, you just delete the entire i -th row and the i -th column from the adjacency matrix A .
- **Link removal:** To remove the connection between node i and node j , you simply set the matrix element a_{ij} to 0. If the graph is undirected (symmetric), you must also remove the reciprocal link by setting a_{ji} to 0.

1.6.1 | Complete Graphs, Complement, K-Cliques and Bipartite Graphs

Let us now discuss some special types of graphs that are commonly studied in graph theory and network science, including complete graphs, graph complements, k-cliques and bipartite graphs.

Definition 1.6 (Complete Graph). A completely connected graph K_N with N nodes with all possible links is called a **complete graph**, meaning every node has a degree of $K = N - 1$:

$$\#l = \binom{N}{2} = \frac{N!}{2!(N-2)!} = \frac{N(N-1)}{2}.$$

Definition 1.7 (Graph Complement and Self-Complementary). The **graph complement** is a network consisting of the exact same set of nodes, but with “opposite” connections. Specifically, two nodes are linked in the complement graph if and only if they were not linked in the original graph.

Algebraically, if A is the adjacency matrix of the original graph, the matrix of its complement can be calculated by applying $(A + 1) \pmod{2}$ to the elements (keeping the diagonal as zero).

Furthermore, a graph is defined as **self-complementary** if it is isomorphic to its own complement. This means that, despite the links being inverted, the new network shares the exact same structural topology as the original one.

Definition 1.8 (K-Clique). A **K-clique** is a fully connected subgraph of G with degree K . Finding a clique of a certain size in a graph, known as the **clique problem**, is NP-complete. This means it is computationally infeasible, as the complexity grows more than a power of the number of nodes.

A strongly connected subgraph, such as a clique, can be seen as a **community** within the network, meaning its nodes are likely to share similar properties. While a single link in an adjacency matrix describes a simple 1-to-1 relationship, a clique can effectively represent an N -body relationship. Practical examples of such structures include protein complexes, chemical reactions, and other multi-body interactions.

Definition 1.9 (Bipartite Graph). A graph that connects two different types of nodes between each other, but NOT between themselves. It is represented by an $M \times N$ rectangular adjacency matrix. In terms of matrix algebra, a bipartite graph is an imprimitive matrix of order 2.

Thus in a bipartite graph, we have two distinct sets of nodes, and links only exist between nodes from different sets. This structure is common in many real-world networks, such as social networks (e.g., people and their affiliations), biological networks (e.g., genes and diseases), and information networks (e.g., users and items in a recommendation system).

In a complete bigraph every node of one type is connected to every node of the other type. The number of links in a biclique is given by $L = M \times N$, where M and N are the number of nodes in the two sets.

The **adjacency matrix** of a bipartite graph can be represented in block form as follows:

$$A = \begin{pmatrix} 0 & B \\ B^T & 0 \end{pmatrix},$$

where B is an $M \times N$ matrix that represents the connections between the two sets of nodes. The upper left and lower right blocks are zero matrices, indicating that there are no connections between nodes of the same type.

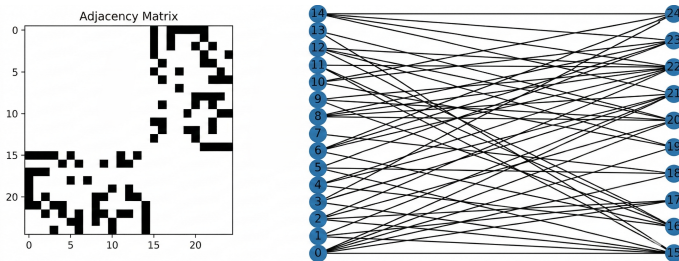


Figure 1.5: An example of a bipartite graph and its corresponding block adjacency matrix; it is structured in a way that reflects the bipartite nature of the graph, with the off-diagonal blocks representing the connections between the two sets of nodes.

Primitivity and Imprimitivity

When representing a bipartite graph with an $(M + N) \times (M + N)$ square adjacency matrix, we can always find a node permutation that neatly separates the two sets of nodes. This rearrangement

leads to a block matrix structure featuring zero blocks on the main diagonal and two non-diagonal blocks representing the cross-connections: there exists a permutation matrix P such that

$$PAP^{-1} = \begin{pmatrix} 0 & B_{mn}^1 \\ B_{nm}^2 & 0 \end{pmatrix}.$$

In terms of matrix algebra, this specific block structure means the bipartite graph corresponds to an **imprimitive matrix of order 2**. Conceptually, traversing this network results in a continuous “flip” in node values as you alternate between the two distinct sets. A **primitive matrix** instead corresponds to a graph where there is a path of any length between any two nodes, without the need for such a flip, indicating a more interconnected structure.

This topological and algebraic concept can be generalized to higher dimensions: an **N -partite graph** corresponds exactly to an **N -imprimitive matrix**. By applying the appropriate node permutation, its adjacency matrix can be organized into a “cyclic” N -block array. In this configuration, all nodes within a given block are connected *only* to nodes belonging to another specific block, and there are no connections between nodes within the same block. This structure leads to a continuous “flip” in node values as you traverse the network, cycling through the different blocks in a specific order.

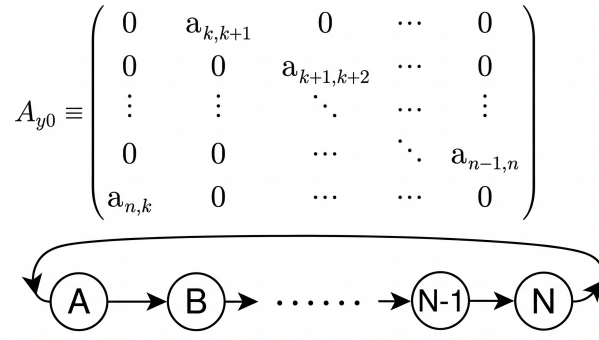


Figure 1.6: An example of a n -partite graph and its corresponding block adjacency matrix; the adjacency matrix is structured in a way that reflects the n -partite nature of the graph, with the off-diagonal blocks representing the connections between the n sets of nodes.

Determining whether a matrix is primitive or imprimitive is crucial in network analysis. **Primitivity** is a fundamental algebraic property required to apply several advanced mathematical theorems to networks, most notably the Perron-Frobenius theorem.

Tensor Contraction

Given the rectangular adjacency matrix B of dimensions $M \times N$, we can perform a tensor contraction to obtain two distinct square networks that connect nodes of the same type. This is achieved through simple matrix multiplication:

- An $M \times M$ matrix calculated as $B \cdot B^T$.
- An $N \times N$ matrix calculated as $B^T \cdot B$.

In both of these resulting matrices, two nodes of the same type are linked if there is at least one path between them through a node of the other type in the original bipartite graph. If we treat the new networks as weighted, the link weight corresponds exactly to the number of common nodes of the other type they share.

Common examples of this projection include co-authorship networks (authors connected by shared papers), actors in the same movie, genes related to the same diseases, or drugs acting on the same diseases.

Remark (Directed Graph Symmetrization). *In addition to the standard topological symmetrization obtained by simply adding the matrix and its transpose ($A + A^T$), a directed graph represented by an $N \times N$ adjacency matrix A can be projected into two distinct symmetric graphs, B and C , using matrix multiplication.*

This process reveals hidden similarities between nodes based on their connection patterns:

- *By computing $B = A^T \cdot A$, we connect two nodes if they both point to at least one common target node. This is widely known as “bibliographic coupling”; for instance, two distinct research papers are linked if they cite the exact same reference in their bibliography.*
- *Conversely, by computing $C = A \cdot A^T$, we connect two nodes if they receive incoming links from at least one common source node. This represents “co-citation coupling”, meaning two papers are linked if they are both cited by the exact same third paper.*

Example (The Human Disease Network). An example of a bipartite graph is the “diseaseome” (e.g., OMIM database), a gene-pathology network. By contracting the matrix, we can obtain a gene-gene network based on shared diseases, or a pathology-pathology network based on shared genes.

Analyzing this projected disease network reveals several key medical insights:

- **Communities and connectivity:** Similar diseases (such as various types of cancers) share many genes and are therefore highly connected. As a result, diseases naturally form clustered communities based on organ-related proximity (e.g., heart, brain, or blood diseases) or functional similarities.
- **Unexpected topological results:** The network structure shows that, counterintuitively, not all organ-related diseases are connected. Furthermore, discovering unexpected links between seemingly unrelated diseases opens the door to **drug repurposing**—suggesting that a drug effective for one disease might also be applied to a connected, yet distinct, pathology.
- **Further analysis:** To extract deeper biological meaning, the network can be enriched by attaching additional “labels” to the nodes, such as disease mortality rates or the specific functional role of a gene within the cell.

1.6.2 | Trees and Stars

Let us now discuss some special types of tree-like structures that are commonly studied in graph theory and network science, including star networks and tree networks.

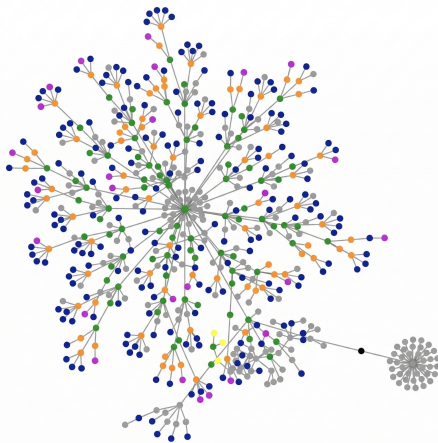
Star Network

The star network is a simple network topology where one central node is connected to all other nodes, which are not connected to each other. This structure resembles a star, with the central node acting as the **hub** and the other nodes as the **spokes**. The star network is characterized by its simplicity and efficiency in terms of communication, as all nodes can communicate with each

other through the central hub. However, it also has a significant drawback: if the central node fails, the entire network becomes disconnected, as there are no alternative paths for communication between the peripheral nodes. The star network is often used in scenarios where a central authority or coordinator is needed, such as in certain types of organizational structures or in some communication networks. However, it is not suitable for large-scale networks or those that require high levels of redundancy and fault tolerance, due to its vulnerability to the failure of the central node.

Tree Network

A tree network is a hierarchical structure that consists of nodes connected in a **parent-child relationship**. It is a type of **acyclic graph** where each node has one parent and can have multiple children. The topmost node is called the root, and the nodes at the bottom are called leaves. Tree networks are commonly used in organizational structures, file systems, and decision-making processes. They allow for efficient data organization and retrieval but can become inefficient if the tree becomes too deep or unbalanced.



It is characterized by its branching structure, where each node can have multiple child nodes, but only one parent node. This structure allows for a single path between any two nodes, making it easy to navigate and manage; there is no redundancy in the connections, which can lead to efficient communication and data transfer. However, it can also lead to bottlenecks if the root node or any intermediate nodes fail, as it can disrupt the entire network.

Figure 1.7: On the left, an example of a tree network, where the central node is the root, and the most external nodes are the leaves. Each node has one parent and can have multiple children, forming a hierarchical structure.

With multiple tree networks, it is possible to create a more complex structure, such as a **forest**, which consists of multiple trees that are not connected to each other. This can provide redundancy and improve the overall resilience of the network.

Example (Fractals as regular trees.). A fractal can be mathematically modeled as a regular tree network. These structures are highly ordered and characterized by specific topological properties: they are self-similar, modular, and deeply hierarchical.

To understand their spatial properties, we can define two metrics for a tree graph:

- **Graph Volume (V):** The total number of nodes in the network.
- **Graph Surface (S):** The number of leaf nodes (nodes with a degree of 1).

In standard Euclidean geometry (or regular lattices), as an object grows, its surface-to-volume ratio tends to zero ($\lim_{N \rightarrow \infty} \frac{S}{V} = 0$). However, fractal networks behave very differently: the

number of leaves grows proportionally to the total number of nodes, meaning the surface is directly comparable to the volume ($V \simeq S$). In practical terms, a huge fraction of a fractal tree's nodes are leaves.

1.6.3 | Directed Acyclic Graph (DAG)

A directed acyclic graph (DAG) is a type of graph that consists of nodes connected by directed edges, where there are no cycles. In a DAG, each edge has a direction, indicating the flow of information or dependencies between nodes. DAGs are commonly used in various applications, such as scheduling tasks, representing dependencies in software development, and modeling hierarchical relationships. It can be used to represent complex systems where there are multiple levels of dependencies, without the risk of creating cycles that can lead to infinite loops or deadlocks.

With DAGs we can represent more complex relationships than trees, as they allow for multiple paths between nodes, establishing an effective hierarchy. However, they can also be more difficult to manage and navigate, especially as the number of nodes and edges increases. DAGs are often used in scenarios where there are multiple dependencies or relationships between entities, such as in project management, data processing pipelines, and knowledge representation.

For networks with these properties, the adjacency matrix is **upper triangular**, which means that all the entries below the main diagonal are zero. This is because in a DAG, there are no cycles, and therefore, there can be no edges that point back to previous nodes. The upper triangular structure of the adjacency matrix reflects the hierarchical nature of the DAG, where nodes can only have edges pointing to nodes that come after them in the hierarchy. Through this matrix, we can represent the structure of the DAG and analyze its properties as a linear algebra problem, such as finding the longest path or determining the reachability of nodes.

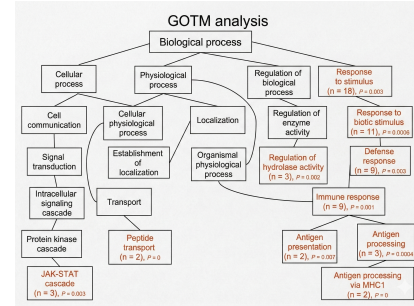


Figure 1.8: An example of a directed acyclic graph (DAG) on the GOTM analysis (which demonstrated that more specific biological processes, such as immune response, regulation of hydrolase activity, peptide transport and JAK-STAT cascade, were significantly modulated by the microbiota; the number of observed genes in a particular biological process is indicated by "n"). The edges have a direction and there are no cycles. The DAG structure allows for multiple paths between nodes, establishing an effective hierarchy.

1.7 | Planarity and Homomorphism

Planarity and homomorphism are important concepts in graph theory that help us understand the structure and properties of networks. They provide insights into how networks can be represented and analyzed.

1.7.1 | Planarity and Genus

A graph is defined as **planar** if it can be drawn on a two-dimensional plane without any of its links (edges) crossing each other.

In topological terms, a planar graph has a **genus** of 0. The genus of a graph represents the minimum number of “holes” (or handles on a mathematical manifold, like a torus or a donut) that a surface must have so that the graph can be drawn on it without any intersecting links.

Because links cannot cross, planar graphs face strict structural limitations and must be relatively sparse. Specifically, there is a rigid upper bound: a planar graph with N nodes can have at most $3N - 6$ links, meaning the number of links scales linearly, $O(N)$.

To determine if a graph is planar, we rely on **Kuratowski’s Theorem**. This theorem states that a graph is planar if and only if it does not contain any subgraphs that are homomorphic (topologically equivalent) to K_5 (the complete graph on 5 nodes) or $K_{3,3}$ (the complete bipartite graph with two sets of 3 nodes).

1.7.2 | Graph Homomorphism

A **graph homomorphism** is a mathematical mapping f from a network G (with N nodes) to a smaller network G' (with $M < N$ nodes) that rigorously preserves node adjacency.

Algebraically, if two nodes u and v are connected in the original graph ($u \sim v$), their mapped counterparts must be connected in the new graph ($f(u) \sim f(v)$):

$$G' = f(G) \quad \text{with} \quad f : G \rightarrow G' \quad | \quad u \sim v \implies f(u) \sim f(v).$$

Practically, a homomorphism acts as a **dimensionality reduction** for a graph. It simplifies the network by drastically reducing the number of nodes while retaining its essential connectivity patterns and macroscopic characteristics. A classic example is mapping a large network of individual nodes into a smaller homomorphic network of **communities**: in this new graph, each super-node represents an entire community, and the links show how different communities interact with one another.

1.8 | Network Reduction and Filtering

We are often interested in reducing the complexity of a network by filtering out less important links, which can help us to focus on the most relevant connections and to analyze the structure of the network more effectively. This process can be done using various methods, such as minimum spanning trees (MST) or link selection rules based on global or local criteria.

1.8.1 | Minimum Spanning Tree (MST)

Let us define a spanning tree as a subgraph of a connected graph that includes all the vertices of the original graph and is a tree (i.e., it is connected and has no cycles) with a special property.

Definition 1.10 (Min/Max Spanning Tree). A minimum/maximum spanning tree (MST) is a subset of the edges of a connected, weighted graph that connects all the vertices together without any cycles and with the minimum/maximum possible total edge weight. It is a method for finding the most efficient way to connect all the nodes in a graph while minimizing/maximizing the total cost: it finds the backbone of the network.

The MST can be used in various applications, such as designing efficient communication networks, clustering data, and solving optimization problems. There are several algorithms for finding the MST, including Prim's algorithm and Kruskal's algorithm, which are based on greedy strategies to build the tree incrementally by selecting the edges with the smallest weights.

Kruskal's Algorithm Kruskal's algorithm is a popular method for finding the minimum spanning tree (MST) of a connected, weighted graph. The algorithm works by sorting all the edges of the graph in non-decreasing order of their weights and then adding edges to the MST one by one, as long as they do not form a cycle. The process continues until there are $V - 1$ edges in the MST, where V is the number of vertices in the graph. To resume:

1. Start: single node and list of all links (edges);
2. Sort all edges in non-decreasing order of their weights;
3. Add edges to the MST one by one, ensuring no cycles are formed;
4. Continue until there are $V - 1$ edges in the MST, where V is the number of vertices.

Kruskal's algorithm is efficient and can be implemented using a disjoint-set data structure to keep track of the connected components of the graph, ensuring that cycles are not formed when adding edges to the MST.

Prim's Algorithm [...]

Example (Pearson Correlation Coefficient). Pearson's correlation matrix (distance $d = 1 - r$, $0 \leq d \leq 2$) is a common way to represent the relationships between nodes in a network, where the correlation coefficient r measures the strength and direction of the linear relationship between two variables. The distance d is calculated as $d = 1 - r$, which transforms the correlation coefficient into a distance metric that can be used to analyze the network structure. In this

representation, a high correlation (close to 1) corresponds to a small distance (close to 0), indicating that the nodes are closely related, while a low correlation (close to -1) corresponds to a large distance (close to 2), indicating that the nodes are not closely related.

It is a fully connected weighed network, which is very difficult to analyze and visualize. Connectivity can be reduced by considering its MST: if we find high correlation between two nodes, we can consider them as connected, otherwise not (correlation is a measure of similarity between two nodes: when the correlation is high, the nodes are similar as $A \equiv B$, $B \equiv C$, then $A \equiv C$ if A and C are also highly correlated). Doing this, we can find the backbone of the network, which is a tree structure that captures the most important relationships between the nodes. This transformation allows us to analyze and visualize the network more effectively, as it reduces the complexity while preserving the essential connections. However, it is important to note that this approach lead to the **loss of some information**, as it removes weaker connections that may still be relevant in certain contexts.

LIMITS: ALL cycles are removed: very drastic transformation

1.8.2 | Link Selection Rules

Global rules for link selection

We can also consider other global rules for link selection, which involve setting a threshold to determine which links to keep in the network. For example:

- consider the total distribution of link weights W and define a cutoff threshold C to keep only $W_{ij} > C$;
- define a parametric link distribution (eg Gaussian, Gamma) and keep only links so that $p(W_{ij}) < p_0$.

But this approach has some limitations, as it can lead to the loss of important information about the network structure:

- Global thresholding methods miss the local relevance of low link values (different weight distribution as a function of local node features such as their degree).

Local rules for link selection

Local methods analyze weights in node neighborhoods Link selection based on single-node null model (pdf)

Example (Serrano Boguna Vespignani PNAS07).

Node of degree K with positive weighted links W_{ij} (normalized to 1, can use absolute value if also negative) Hypothesis: weights are extracted from a random split of unit segment $[0:1]$ in K parts (geometric distribution $G(K)$) Keep only links for which $G(W_{ij}) < p_0$ (global threshold but applied to local node-dependent distribution)

1.9 | Spectral Analysis of Networks

Through the analysis of the eigenvalues and eigenvectors of matrices associated with a network, such as the adjacency matrix or the Laplacian matrix, we can gain insights into the structural properties of the network. Spectral analysis can reveal important information about the connectivity, community structure, and dynamics of the network. For example, the largest eigenvalue of the adjacency matrix can indicate the presence of hubs in the network, while the second smallest eigenvalue of the Laplacian matrix can provide information about the connectivity and robustness of the network. Spectral methods are widely used in various applications, including clustering, dimensionality reduction, and network visualization.

1.9.1 | The Eigenvalue Spectrum

We can read the problem as an **eigenvalue problem**:

$$A \cdot \mathbf{x} = \lambda \cdot \mathbf{x},$$

where A is the adjacency matrix of the network, $\mathbf{x}$ is an eigenvector, and λ is the corresponding eigenvalue. The eigenvalues and eigenvectors can provide insights into the structure and properties of the network, such as identifying communities, detecting central nodes, and understanding the dynamics of processes on the network. Spectral analysis can also be used to identify important features of the network, such as its connectivity, robustness, and resilience to perturbations.

Example (Principal Component Analysis (PCA)). Principal Component Analysis (PCA) is a dimensionality reduction technique that can be applied to network data to identify the most important features or components that capture the variance in the data. By analyzing the eigenvalues and eigenvectors of the covariance matrix of the network data, PCA can help to identify patterns and relationships between nodes, as well as reduce the complexity of the data while preserving its essential structure. PCA can be particularly useful for visualizing high-dimensional network data and for identifying clusters or communities within the network.

We can also imply the **characteristic polynomial** of the adjacency matrix A , defined as

$$\Phi(\lambda) = \det(A - \lambda I),$$

where I is the identity matrix. The roots of this polynomial are the eigenvalues of A , and analyzing them can provide further insights into the structural properties of the network.

1.9.2 | Powers of the Adjacency Matrix

Let us start by defining the powers of the adjacency matrix A . The n -th power of the adjacency matrix, denoted as A^n , can be computed using matrix multiplication. The entry $(A^n)_{ij}$ represents the number of paths of length n from node i to node j in the network:

$$B = A^2, \quad B_{ij} = \sum_k A_{ik} A_{kj}, \quad B_{ii} = \sum_k A_{ik} A_{ki}.$$

This is because each multiplication of the adjacency matrix counts the number of ways to traverse from one node to another through intermediate nodes. This multiplication tells us how many paths

of length n exist between nodes i and j . For example, if $A_{ij}^2 > 0$, it means there is at least one path of length 2 (since we are looking at the second power) connecting nodes i and j .

B_{ii} can infer on directed and undirected paths in the network, as it counts the number of paths that start and end at the same node i . In an **undirected network**, this would count the number of loops or cycles that involve node i , while in a **directed network**, it would count the number of directed paths that return to node i . This information can be useful for understanding the local structure and connectivity of the network around node i .

- The first non-null term ij of the k -th power of A indicates the length of the shortest path from i to j (without specifying the route(s)).
- The k -th power matrix counts the number of walks from one node to another.
- If an element $A_{ij}^k = 0$ for each k , there are no paths between the two nodes (up to $k_{\max} = \text{\#nodes}$).
- If the graph is directed, certain paths/walks may not exist.
- If no paths exist between two nodes in a symmetric graph, then the network is disconnected.

1.9.3 | Spectra of Specific Graphs

Cycles A *directed cycle* is a path that starts and ends at the same node, following the direction of the edges

$$(C_{ND})^N = \mathbb{I} \implies \lambda_i^N = 1 \implies \lambda_i = e^{2\pi i k/N} \quad \text{for } k = 0, 1, \dots, N-1,$$

showing that the eigenvalues of the adjacency matrix of a directed cycle graph are the N -th roots of unity. This means that the eigenvalues lie on the unit circle in the complex plane, and their distribution can provide insights into the structure and properties of the directed cycle graph. The presence of cycles in a network can have significant implications for its dynamics and stability, as they can lead to feedback loops and complex interactions between nodes. For example, if $N = 4$ we have:

$$C_{4D} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

[...]

Completely connected graphs These graphs have a simple structure, where every node is connected to every other node. The adjacency matrix of a completely connected graph with N nodes can be represented as:

all-ones matrix, rank 1, with eigenvalues N and 0 (with multiplicity $N - 1$)

If we remove the self-loops (i.e., the diagonal entries), we get a traceless matrix with $N - 1$ entries

equal to 1 in each row, which can be represented as:

$$A = \begin{pmatrix} 0 & 1 & 1 & \cdots & 1 \\ 1 & 0 & 1 & \cdots & 1 \\ 1 & 1 & 0 & \cdots & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \cdots & 0 \end{pmatrix},$$

which is traceless and has $N - 1$ entries equal to 1 in each row (since each node is connected to $N - 1$ other nodes). The diagonal entries are zero because there are no self-loops in the graph. This structure leads to a highly symmetric adjacency matrix, which has implications for its spectral properties and the overall connectivity of the network.

The eigenvalues of this adjacency matrix can be calculated, and they consist of one eigenvalue equal to $N - 1$ (which corresponds to the eigenvector with all entries equal) and $N - 1$ eigenvalues equal to -1 (which correspond to the eigenvectors that are orthogonal to the all-ones vector). This spectral property reflects the high level of connectivity in the completely connected graph, where each node is directly connected to every other node. The presence of a single large eigenvalue indicates that there is a strong overall connection in the network, while the negative eigenvalues indicate that there are no other significant structures or communities within the network.

Bipartite graphs

DAG

Note on Cycles and Network Sparsity

If a network has no cycles there is a single path between any two nodes (which is the shortest path necessarily): it is a powerful approximation for many real networks, which are often very sparse (e.g. with a small number of links as compared to the number of nodes $o(N^2)$). If a network is sufficiently sparse (e.g. with a small number of links as compared to the number of nodes $o(N^2)$) it is very likely not to have cycles, or they can be considered negligible (e.g. in generating multiple paths between nodes): we can consider the network as a **tree**, or more generally as a DAG (if directed), without cycles.

This is an approximation used very often in theorems, in particular for random networks ensembles (see following lectures) In sparse networks cycles are rare, and long cycles even more.

Thus the presence of cycles is an indicator of a particular network design (that cannot be obtained by randomly connecting nodes, see following lectures on network entropy)

What happens to the spectrum of these matrices if we perturb slightly the network structure? For example, if we add or remove a few edges or loops (cyclicality, bipartition, all-ones...), how does this affect the eigenvalues and eigenvectors of the adjacency matrix? This question is important for understanding the stability and robustness of the network, as well as for analyzing the dynamics of processes that occur on the network. Perturbations can lead to changes in the spectral properties of the network, which can in turn affect its overall behavior and functionality. They can be studied by analyzing the sensitivity of the eigenvalues and eigenvectors to changes in the adjacency matrix, generating random matrices and analyzing their spectra, or by using perturbation theory

to understand how small changes in the network structure can lead to changes in its spectral properties (like adding random links or random weights to the adjacency matrix). This analysis can provide insights into the resilience of the network to perturbations and its ability to maintain its functionality under changing conditions.

1.9.4 | Isomorphism and Co-spectrality

Two graphs are isomorphic if they are equal up to node permutation (transformation of similitude in matrix algebra)

$$A_1 = P A_2 P^T, \quad (P P^T = \mathbb{I})$$

$$|A_1| = |P A_2 P^T| \implies |A_1| = |A_2|,$$

thus isomorphic graphs have the same spectrum, but the converse is not necessarily true: two non-isomorphic graphs can have the same spectrum (called **cospectral** graphs):

$$\begin{aligned} \text{Isomorphic} &\implies \text{Cospectral}, \\ \text{Non-Cospectral} &\implies \text{Non-Isomorphic}. \end{aligned}$$

This means that while the spectrum of a graph can provide important information about its structure and properties, it is not a complete invariant for graph isomorphism. Therefore, additional methods or invariants may be needed to determine whether two graphs are isomorphic or not, especially in cases where they are cospectral.

But co-spectral DOES NOT imply isomorphic! (for $N > 4$ knots) [Can one hear the shape of a drum? M. Kac Am Month. 73 (4), 1966]

Let us resume some of the key points about the spectrum of a graph:

- Eigenvalue spectrum: "Summary", "modes" of a graph, tells us about the structure of the graph, but not all the information (e.g. it does not tell us about the specific arrangement of nodes and edges, or about the presence of certain substructures or motifs in the graph)
- Spectrum is independent of representation (invariant for node re-indexing)
- It is invariant for permutation operation: being a similarity transform (involving a matrix and its inverse) the eigenvalue problem is identical (related to the properties of the determinant of matrix products) problem is identical (related to the properties of the determinant of matrix products)
- There are few exact results and some numerical results for particular cases (more on trees, less on general graphs)

Co-spectrality

1.9.5 | Graph Distances

Once the eigenvalues have been sorted (with the addition of zeros for networks with different number of nodes, corresponding to isolated nodes that do not contribute to spectrum calculation)

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_N,$$

$$\mu_1 \geq \mu_2 \geq \dots \geq \mu_N, d(G_1, G_2) = \sqrt{\sum_{i=1}^N (\lambda_i - \mu_i)^2},$$

we can define a spectral distance between two graphs G_1 and G_2 as the Euclidean distance between their sorted eigenvalue spectra. This distance provides a measure of how similar or different the two graphs are in terms of their spectral properties. A smaller distance indicates that the graphs have more similar spectral characteristics, while a larger distance indicates that they are more different. This spectral distance can be used for various applications, such as clustering graphs, comparing network structures, and analyzing the evolution of networks over time. However, it is important to note that this distance is not a complete invariant for graph isomorphism, as two non-isomorphic graphs can have the same spectrum (co-spectral graphs). Therefore, while the spectral distance can provide useful insights into the similarity of graphs, it should be used in conjunction with other methods for a more comprehensive analysis of graph structure and properties.

Similar Graphs and Their Spectra

To define similar graphs we have to define a distance between graphs, which can be done using the spectral distance defined above. Similar graphs are those that have a small spectral distance, indicating that their eigenvalue spectra are close to each other. This means that the structural properties of the graphs are similar, as reflected in their spectra. Similar graphs may have similar connectivity patterns, community structures, or other structural features that contribute to their spectral similarity. However, it is important to note that similar graphs may not necessarily be isomorphic, as they can have different arrangements of nodes and edges while still having similar spectral properties. Therefore, while the spectral distance can provide insights into the similarity of graphs, it should be used in conjunction with other methods for a more comprehensive analysis of graph structure and properties.

There seems to be a strong correlation between spectral distance and graph edit distance. Thus proximity of spectra reflects graphs with similar structure (even if exceptions exist)

TODO: insert graphs

Graph edit distance Graph edit distance is a generalization of edit distance between strings of symbols, which is used in bioinformatics or in text analysis to identify similarity in different strings (protein or DNA sequences, words or periods). In symbol sequences, we can have elementary operations like symbol substitution, symbol insertion or symbol removal.

The set of operations that connect one string to the other can be represented in a graph table, with a path representing the editing steps. Different scores can be defined to identify the best matching between two strings (and thus between two graphs), for example

- Longest common subsequence (subgraph)
- Global alignment scores based on different values of penalties associated to mismatches, insertions, deletions

Example (DNA sequences). For proteins and DNA sequences there is a natural node ordering, thus edit distance is simpler to calculate (substitution/insertion/deletion along the sequences). For networks there is no natural node ordering, thus also the starting point of node-to-node mapping is nontrivial. Possible approaches (open problems):

- Spectral approaches to matrices (eigenvectors components and node similarities)
- Similarity between node embeddings

2 | Random Graph Ensembles

In this chapter, we will explore the concept of random graph ensembles, which are fundamental in understanding the properties and behaviors of complex networks. We will start with the simplest model, the Erdős-Rényi random graph, and then move on to more complex models such as the Stochastic Block Model (SBM). We will also discuss the thermodynamic limit, degree distribution, phase transitions, and other properties of these random graph ensembles.

2.1 | Erdős-Rényi Random Graphs

We are going to start with the simplest model of random graphs, which is the Erdős-Rényi (E-R) model. This model serves as a fundamental null model for understanding the properties of complex networks.

Definition 2.1 (Erdős-Rényi Random Graphs). Networks whose interactions (links) between nodes are chosen randomly, without a priori correlations or rules. In statistical mechanics terms, it is comparable to a “perfect gas” with molecules (nodes) experiencing random “sparse” impacts (links).

It can be seen as a **null model** for complex networks, as it represents a system with no structure or correlations.

If we expect to find a certain property in a real network, we can compare it to the expected value of that property in an E-R random graph. If the observed value significantly deviates from the expected value, we can infer that there is some underlying structure or correlation in the real network that is not captured by the E-R model. It can also be used to confront different systems, such as comparing the structure of a social network to that of a biological network, to identify unique features and commonalities.

If we make the statistical analogy with a gas, the E-R model represents a perfect gas, where the nodes are like molecules that interact randomly without any specific rules or correlations. The edges, which are generated randomly, represent some average quantities of the system, not the distance between nodes or the strength of interactions. The E-R model is a useful starting point for understanding the properties of complex networks, but it is important to note that real-world networks often exhibit more complex structures and correlations that are not captured by this simple model.

2.2 | Statistical Ensembles of Networks

If we imagine to take a picture of our ideal gas system, we would see a random distribution of molecules (nodes) and their interactions (links, diluted interactions, since we are speaking of ideal gas). The E-R model captures this randomness and serves as a baseline for understanding the properties of complex networks. In the next instant, the picture would change, and we would see a different random distribution of molecules and interactions, but the overall properties of the system would remain the same, as they are determined by the average quantities rather than the specific configuration of nodes and links. We will see that these analogies with ensembles will be very useful to understand the properties of complex networks, and also the E-R model will maintain some properties constant even if we change the specific configuration of nodes and links, which is a key feature of random graph ensembles.

We are going to use **ensembles of networks** with fixed parameters: same macroscopic state (e.g. link density, degree pdf) different microstates (single-node properties), and we will see that the properties of the ensemble are determined by the average quantities rather than the specific configuration of nodes and links. The definition of the ensemble will suggest a construction rule to generate the microstates of the ensemble and to compute the probability of generating a specific microstate.

Remark. *The number of ALL symmetric networks (microstates) with N nodes (whatever the links) is:*

$$2^{\frac{N(N-1)}{2}}, \quad \sum_{i=1}^N \binom{N}{i} = 2^N.$$

This means that the number of possible configurations of links between N nodes is exponential in the number of nodes, which highlights the complexity of the space of possible networks and the importance of using ensembles to understand their properties.

The most common ensembles are the microcanonical ensemble, where we fix the number of links, and the canonical ensemble, where we fix the probability of having a link between two nodes. Let us introduce these two ensembles in more detail.

2.2.1 | Microcanonical Ensemble $G_{N,M}$

Definition 2.2 (Microcanonical Ensemble). An ensemble of networks with N nodes, representing a fixed system size, and M links, representing the system energy. The links are randomly selected from the possible $N(N-1)/2$ for a symmetric network or $N(N-1)$ for a directed graph.

The idea behind a microcanonical ensemble is that we are looking at all possible configurations of a network with a fixed number of nodes and links. Each configuration (or microstate) is equally likely, and the properties of the ensemble are determined by the average quantities rather than the specific configuration of nodes and links.

Looking at the properties of the microcanonical ensemble, we can see that it is a very simple model, but it can still capture some important properties:

- The N nodes represent the system size, which is fixed.
- The M links represent the system energy, which is also fixed.

- The links are randomly selected from the possible $N(N - 1)/2$ for a symmetric network or $N(N - 1)$ for a directed graph, which means that there is no specific structure or correlation in the network.

So we have

$$M_{max} = \frac{N(N - 1)}{2}, \quad \#G_{N,M} = \binom{M_{max}}{M},$$

such that for a fixed number of nodes N and links M , the number of possible configurations (or microstates) of the network is given by the binomial coefficient $\binom{M_{max}}{M}$, which counts the number of ways to choose M links from the total possible M_{max} links. This highlights the combinatorial nature of the microcanonical ensemble and the importance of using statistical methods to understand its properties.

Remark (Construction Rule). *The construction rule of one instance of the ensemble is:*

1. Take N nodes.
2. Repeat M times: choose 2 nodes randomly and connect them, checking for loops or existing links.

The probability of generating “bad” links, such as loops and multilinks, is about $1/N$. The effect of this selection becomes negligible for large N .

The probability of generating a loop is $1/N^2$, and the probability of generating a multilink is M/N^2 . In order to connect two times the same nodes, the probability is $1/N^2$. For large N , i.e. in the thermodynamic limit, these probabilities become negligible, and the microcanonical ensemble can be effectively generated by randomly selecting M links from the possible $N(N - 1)/2$ links without worrying about loops or multilinks.

2.2.2 | Canonical Ensemble $G_{N,P}$

Often it is more convenient to **fix the probability** of having a link between two nodes, rather than the number of links. This leads us to the canonical ensemble.

Definition 2.3 (Canonical Ensemble). An ensemble of networks with N nodes (fixed system size) and a fixed probability P to have a link between them, representing a fixed average energy or temperature.

This means that in the canonical ensemble, given the nodes and the probability P , we can generate a random graph by connecting each pair of nodes with a link with probability P . The properties of the ensemble are determined by the average quantities:

$$P = \langle E \rangle \propto T, \quad \langle l \rangle = P \frac{N(N - 1)}{2}.$$

Binomial Link Distribution

Given N and P I do not have a fixed number of networks belonging to the canonical ensemble Better: ALL graphs with N nodes belong to $G_{N,P}$ with a different probability depending on P The probability of having a graph with m links and N nodes is governed by the binomial distribution:

$$P(m) = \binom{N(N - 1)/2}{m} P^m (1 - P)^{N(N - 1)/2 - m}$$

The average grows as the ...

Remark (Construction Rule). *The probability of having a graph with m links and N nodes is governed by the binomial distribution. To construct an undirected graph:*

- Take $N(N-1)/2$ values A_{ij} uniformly distributed between 0 and 1.
- If $A_{ij} > 1 - P$ then $A_{ij} = 1$, else $A_{ij} = 0$.
- Apply symmetrization using logic “OR”: $A \vee A^T$.

If the network is sufficiently large, the probability of having a graph with m links and N nodes is governed by the binomial distribution, which can be approximated by a Poisson distribution in the limit of large N and small P with $P \cdot N$ kept constant. The average number of links in the canonical ensemble is given by $\langle l \rangle = P \frac{N(N-1)}{2}$, which grows as the square of the number of nodes for fixed P . This highlights the fact that in the canonical ensemble, the number of links can vary widely depending on the specific configuration of nodes and links, but the average properties of the ensemble are determined by the probability P and the number of nodes N .

2.2.3 | Equivalence of Ensembles

For $N \gg 1$, the canonical link distribution tends to a Dirac’s delta function. All instances of the canonical ensemble tend to have a number of links equal to the average link number, thus recovering the microcanonical property.

[...]

2.2.4 | Degree Probability Density Function (pdf)

The probability of having a node with k links is the probability of randomly choosing k links among $N-1$:

$$p(k) = \binom{N-1}{k} P^k (1-P)^{N-1-k},$$

Balls and Holes

If we consider the problem of distributing k links (balls-couples) among $N-1$ possible connections (holes), we can see that the degree distribution follows a **binomial distribution**, which can be approximated by a Poisson distribution in the limit of large N and small P with $P \cdot N$ kept constant. This analogy helps us understand the degree distribution in random graphs and how it arises from the random process of connecting nodes with links.

The degree distribution in the canonical ensemble is given by the binomial distribution, which can be approximated by a Poisson distribution in the limit of large N and small P with $P \cdot N$ kept constant

$$p(k) = \binom{N-1}{k} P^k (1-P)^{N-1-k}$$

The average node degree $\langle k \rangle$ is given by $P(N-1)$, which grows linearly with the number of nodes for fixed P :

$$\langle k \rangle = \frac{\langle \# \text{links} \rangle}{N} = \frac{2P \frac{N(N-1)}{2}}{N} = P(N-1) \approx PN,$$

while the average link density is

$$p \frac{N(N-1)}{2}.$$

This highlights the fact that in the canonical ensemble, the degree distribution can vary widely depending on the specific configuration of nodes and links, but the average properties of the ensemble are determined by the probability P and the number of nodes N .

Remark (Bernoulli to Poisson to Delta). *In the thermodynamic limit ($N \rightarrow \infty$, $P \rightarrow 0$) with $P \cdot N = \lambda$ kept constant, the distribution becomes a Poisson distribution:*

$$p(k) \approx \frac{\lambda^k}{k!} e^{-\lambda}$$

In this limit, a random Bernoulli graph tends to a regular graph with equal connectivity for all nodes, even if node proximity is not regular as in a lattice. The Erdős-Rényi model tends to an infinite-dimensional regular lattice.

$$P \cdot N = \lambda = \text{constant}(P \rightarrow 0, N \rightarrow \infty),$$

$$p(k) = \frac{\lambda^k}{k!} e^{-\lambda}, \quad \langle k \rangle = \lambda, \quad \sigma = \sqrt{\lambda},$$

$$\lim_{\lambda \rightarrow \infty} p(k) = N(\lambda, \sqrt{\lambda}) \rightarrow \delta(x - \lambda).$$

[...]

2.3 | Phase Transitions and the Giant Component

Considering the probability of having a property in an ensemble with N nodes

$$P = P(N),$$

depending on the trend we get behaviors that lead to have certain properties valid for almost all graphs or almost none. This is the idea of a phase transition, where we have a critical point that separates two different phases of the system. In the context of random graphs, one of the most important properties is the emergence of a giant component, which is a connected component that contains a finite fraction of the nodes in the graph. The emergence of a giant component is a phase transition that occurs at a critical value of the average connectivity λ .

2.3.1 | Average Connectivity

In E-R networks, the *threshold for transitions depends on the network size N and the connection probability p* . We use the average connectivity λ as the control parameter:

$$\langle k \rangle \approx pN = \lambda$$

2.3.2 | The Giant Component

Definition 2.4 (Giant Component Emergence). • For $\lambda < 1$, there is no giant component.

- For $\lambda \geq 1$, there is a giant component that, for increasing λ , tends to include all the nodes.

Remark (Transcendental Equation). *If we define u as the probability of a node not belonging to the Giant Component, the average field equation yields the transcendental equation:*

$$u = e^{\lambda(u-1)}$$

The order parameter S , which is the fraction of nodes belonging to the giant component, is $S = 1 - u$.

... GC order parameter...

One can find similarities with the **percolation model**: If we have a lattice of nodes and we randomly occupy the links with probability p , we can see that there is a critical value of p above which a giant component emerges, which is similar to the phase transition in random graphs. The percolation model can be seen as a special case of the random graph model, where the nodes are arranged in a lattice and the links are occupied with a certain probability.

Adding one link at a time creates larger and larger clusters, which is known as the “**avalanche phenomenon**”. This is similar to the process of percolation, where adding one link can lead to the emergence of a giant component that contains a finite fraction of the nodes in the graph: after a critical point, the network becomes traversable, connected. The phase transition in random graphs can be understood as a percolation process, where the control parameter is the average connectivity λ .

2.4 | Network Properties in Random Graphs

We can expect that in a random graph, the properties of the network will be determined by the average quantities rather than the specific configuration of nodes and links. This means that we can use statistical methods to understand the properties of random graphs and how they arise from the random process of connecting nodes with links.

2.4.1 | Diameter and Average Path Length

Assuming a regular graph ($N \gg 1$) and a sparse network (regular tree, no cycles, so that from a node there is a unique path to any other node and a very small probability of coming back to the starting node) with

$$\#l \sim O(N) = o(N^2),$$

we can define the **diameter** of the graph as the longest shortest path between any two nodes.

Starting from a node, one can reach $\langle k \rangle$ nodes in one step, $\langle k \rangle^2$ nodes in two steps, and so on. After d steps, one can reach $\langle k \rangle^d$ nodes. Therefore, the diameter of the graph can be estimated by solving the equation:

$$\langle k \rangle^d \sim N.$$

In a random graph, the diameter grows logarithmically with the number of nodes, which means that even for large networks, the distance between nodes remains relatively small. This is a key feature of random graphs and is often referred to as the “small-world” property.

Let us give some insight into the solution of that equation. We can take the logarithm of both sides to get:

$$\dots$$

A similar result holds for the **average path length**

$$\langle l(N) \rangle \propto \ln N.$$

As the network size grows, the distance between nodes grows slowly, which for a D-lattice, translates in the diameter growing as $N^{1/D}$, which is much faster than logarithmic growth.

Random networks are **traversable**, meaning that there is a path between any two nodes, and the distance between nodes grows logarithmically with the number of nodes. This is a key feature, a trivial property of random graphs and is often referred to as the “**small-world**” property, which has important implications for the dynamics and functionality of complex networks.

2.4.2 | Degree Correlations

Let us study the relationship between the degrees of neighboring nodes in a random graph. In an E-R random graph, there is no correlation between the degrees of neighboring nodes, which means that the degree of a node does not depend on the degree of its neighbors. This is because the links are generated randomly, and there is no specific structure or correlation in the network. If the nodes are correlated, we would expect to see a relationship between the degrees of neighboring nodes, such as a positive correlation (where high-degree nodes tend to be connected to other high-degree nodes) or a negative correlation (where high-degree nodes tend to be connected to

low-degree nodes). However, in an E-R random graph, there is no such correlation, and the degree of a node is independent of the degree of its neighbors.

There is an **absence of correlations** between node features. Specifically, there is ALMOST none for finite-size graphs and none in the thermodynamic limit ($N \rightarrow \infty$ and $\#l = O(N)$). For undirected graphs:

$$p(k_1, k_2) \approx p(k_1) \cdot p(k_2).$$

In-Degree and Out-Degree Correlations

Similarly, in an ER directed network, there is no correlation between in-degree and out-degree: $p(k_{IN}, k_{OUT}) = p(k_{IN}) \cdot p(k_{OUT})$. This means that the in-degree of a node does not depend on its out-degree, and vice versa. This lack of correlation is a key feature of random graphs and distinguishes them from more structured networks, where there may be correlations between the degrees of neighboring nodes.

2.5 | Stochastic Block Model (SBM)

Another important model of random graphs is the Stochastic Block Model (SBM), which is a more complex null model that can modelize a nontrivial community structure of the network. It can be obtained by combining diagonal E-R blocks, possibly of different sizes and link densities, with non-diagonal blocks of random links. The diagonal blocks represent the communities, where nodes within the same community are more likely to be connected, while the non-diagonal blocks represent the connections between different communities, which can be random or have a different link density.

Definition 2.5 (Stochastic Block Model). A more complex null model that can modelize a non-trivial community structure of the network. It can be obtained by combining diagonal E-R blocks, possibly of different sizes and link densities, with non-diagonal blocks of random links.

The main algorithm backbone is the same to generate symmetric or directed E-R networks of specific size and link density. The non-diagonal blocks connecting communities of different size (e.g. with N and M nodes) will be rectangular ($N \times M$).

What makes the SBM more complex than the E-R model is that it can capture the community structure of real-world networks, where nodes are organized into groups or communities that have a higher density of connections within the group than between groups. This makes the SBM a more realistic model for understanding the properties of complex networks, as many real-world networks exhibit community structure.

We can play with the structure of the blocks to generate different types of networks, such as assortative networks (where nodes within the same community are more likely to be connected) or disassortative networks (where nodes within the same community are less likely to be connected). We can model networks with a core-periphery structure, where there is a dense core of nodes that are highly connected to each other and a periphery of nodes that are less connected. We can also model networks with a hierarchical structure, where there are multiple levels of communities nested within each other. The SBM is a powerful tool for understanding the properties of complex networks and can be used to generate synthetic networks that mimic the structure of real-world networks.

3 | Spectral Analysis of Random Graphs

We are going to analyze the spectral properties of random graphs, which are important for understanding the dynamics and functionality of complex networks. We will start with the spectral density of the adjacency matrix, which is a key tool for understanding the structure of random graphs. We will then explore the eigenvalue distribution and its implications for the stability and dynamics of random graphs.

The **observables** of interest in the study of random graphs include

- Eigenvalue distribution of the adjacency matrix, which can provide insights into the structure and dynamics of the network.
- Maximum eigenvalue, which can be related to the stability of the network and the presence of a giant component.
- Nearest neighbor spacing distribution (NNSD), which can provide insights into the correlations between eigenvalues and the presence of community structure in the network.

These spectral properties can be analyzed using tools from random matrix theory, which provides a framework for understanding the statistical properties of eigenvalues in large random matrices. By studying the spectral properties of random graphs, we can gain insights into the structure and dynamics of complex networks, and how they differ from more structured networks with community structure or other correlations. Our study cases will be:

- Random symmetric matrix, with entries drawn from a Gaussian distribution, which can be used to model the adjacency matrix of an E-R random graph.
- E-R matrix, which is a specific case of a random symmetric matrix where the entries are binary (0 or 1) and represent the presence or absence of links between nodes in an E-R random graph.
- Hi-C matrix, which is a specific case of a random symmetric matrix that represents the contact frequencies between different regions of the genome in a Hi-C experiment. The spectral properties of Hi-C matrices can provide insights into the 3D organization of the genome and how it relates to gene regulation and other cellular processes.

eigenvalue spectrum of network

For a random symmetric $N \times N$ matrix with elements sampled from a Gaussian pdf $N(0, \sigma)$ the eigenvalue spectrum has the following form. Real spectrum: semicircle centered around origin with a radius depending on node number N and Gaussian SD σ :

$$\rho(\lambda) = \frac{1}{2\pi N \sigma^2} \sqrt{4N\sigma^2 - \lambda^2}, \quad |\lambda| < 2\sigma\sqrt{N} \dots$$

ALL normalized eigenvector components are distributed as a Gaussian pdf: ALL eigenvectors are EQUAL and “poorly informative”

There will be no subsets of nodes particularly emerging from components of our eigenvectors, as they are all equally distributed. This is a key feature of random graphs, where there is no specific structure or correlation in the network, and the eigenvectors do not provide any meaningful information about the structure of the graph.

From the central limit theorem, if we have set of randomly distributed variables, their sum will tend to a Gaussian distribution, regardless of the original distribution of the variables. In the case of random graphs, the eigenvectors are linear combinations of the entries of the adjacency matrix, which are randomly distributed. Therefore, the components of the eigenvectors will also be randomly distributed and will tend to a Gaussian distribution due to the central limit theorem. This means that all eigenvectors will be equally distributed and will not provide any meaningful information about the structure of the graph, which is a key feature of random graphs.

E-R Spectrum

If we take an E-R networks with 0-1 links maximally distributed in a random way, wigner semicircle law will still hold, but with a different radius depending on the link density. It is not exactly the gaussian orthogonal ensemble, but it is still a random symmetric matrix with entries that are randomly distributed. The eigenvalue spectrum of an E-R random graph will still follow the Wigner semicircle law, but the radius of the semicircle will depend on the link density of the graph, and we have a big eigenvalue $\lambda_{\max}$ that emerges from the bulk of the spectrum, which is related to the presence of a giant component in the graph. The presence of this large eigenvalue is a key feature of E-R random graphs and distinguishes them from more structured networks, where there may be multiple large eigenvalues corresponding to different communities or other structures in the graph.

The **maximum eigenvalue** of the adjacency matrix of a network is bound by the average degree $\langle k \rangle$ and the largest degree $k_{\max}$ of the corresponding network as follows:

$$\langle k \rangle \leq \lambda_{\max} \leq k_{\max}.$$

The largest eigenvalue has emerged as a key quantity for the study of a variety of systems and processes, such as virus spread, synchronization of coupled oscillators and stability of couplings in the brain.

Appendices

A | Notation and Conventions

B | Computations and further developments